



UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

Escuela de Postgrado

Ingeniería Civil

APLICACIONES DE IA EN ESTRUCTURAS

ENTREGA FINAL – GRUPO 03

Profesor: Ing. Soncco Sinchi Kurt Walter

Alumnos:

Carballo Retana, David Roberto

Carpio Espino, Edson Gabriel

Ojeda Canales Henry Gabriel

Vela Davila, Ronie

Fecha: 07/10/2026

Desarrollo algorítmico y redescubrimiento físico de la Norma E.030

1. Introducción

El proyecto inició con un enfoque tradicional de Machine Learning Supervisado: proveer a un algoritmo de una tabla de datos sísmicos para que aprendiera a predecir el cortante basal y las derivas. Sin embargo, las rigurosas auditorías académicas revelaron una falencia conceptual crítica: si el dataset es sintético (generado por fórmulas conocidas), la Inteligencia Artificial simplemente 'memoriza' la ecuación. Esto, conocido científicamente como Fuga de Información (Data Leakage), resulta en modelos con 100% de exactitud que carecen de valor científico real, ya que no están prediciendo.

Ante esta problemática, el equipo de investigación decidió orientar el estudio hacia el Aprendizaje Automático Físicamente Informado (PIML), bajo el paradigma de Veridical Data Science. Este enfoque busca trascender la capacidad predictiva convencional de los modelos de Machine Learning para analizar si, a partir de datos estructurales y sísmicos potencialmente ruidosos, los algoritmos son capaces de identificar relaciones y patrones físicamente coherentes. De este modo, la investigación plantea como interrogante

central si un modelo de Machine Learning, sin incorporar de forma explícita las relaciones normativas de la ingeniería sismorresistente, puede inferir y reproducir los patrones fundamentales establecidos por la Norma E.030.

Este replanteamiento masivo ('El Camino B') requirió purgar las variables correlacionadas, implementar clasificadores ordinales para la gestión asimétrica del riesgo, y utilizar programación genética evolutiva (PySR) junto con Teoría de Juegos (SHAP) para auditar el razonamiento matemático de la máquina.

2. Justificación del Triage sísmico

El análisis estructural moderno se basa en modelamientos de Elementos Finitos (FEM) y análisis dinámicos modales espectrales. Aunque precisos, estos métodos exigen tiempos de modelado y cómputo masivos.

La necesidad industrial actual exige un 'Triage Estructural Conceptual'. En medicina, el Triage es la clasificación rápida de pacientes según la gravedad de su herida. En ingeniería, proponemos un Triage Sísmico: la capacidad de evaluar miles de edificaciones a nivel urbano en milisegundos, prediciendo su nivel de

vulnerabilidad usando exclusivamente datos de pre-dimensionamiento preliminar (Zona, Perfil de Suelo, Altura, Sistema Resistente), sin requerir análisis modales. Esta memoria de cálculo documenta la construcción paso a paso del 'cerebro' detrás de este Triage.

3. Manual de Librerías y Funciones Computacionales

El ecosistema de programación se basó íntegramente en Python y Julia. A continuación, se detalla el propósito estructural y las funciones algorítmicas exactas utilizadas por el equipo:

3.1. Pandas y NumPy (Motor de Datos)

Pandas es el estándar de la industria para manipulación de DataFrames tabulares, y NumPy es el motor de álgebra lineal de alto rendimiento en C/C++.

- `pd.DataFrame()` y `pd.read_csv()`: Utilizados para instanciar y leer la matriz paramétrica de 2,500 edificaciones en la memoria RAM del sistema.
- `df.drop(columns=[])`: Función crítica utilizada para purgar la multicolinealidad y la fuga de información, eliminando del tensor de entrenamiento variables perjudiciales.
- `np.random.uniform()` y `np.random.randint()`: Empleados en el script generador para simular el comportamiento estocástico de las

variables (ej. alturas aleatorias entre 2 y 20 pisos).

3.2. Scikit-Learn (Matemática No Supervisada)

Librería esencial para el procesamiento estadístico previo al modelado.

- `StandardScaler.fit_transform()`: Aplica el Z-Score. A cada variable se le resta la media y se divide por la desviación estándar. Esto evita que la IA piense que el 'Peso' (6000 Ton) es más importante que el 'Factor Z' (0.45) solo por tener más ceros.
- `PCA(n_components=2)`: Análisis de Componentes Principales. Calcula la matriz de covarianza y extrae los eigenvectores (vectores propios) para colapsar 37 variables físicas en un gráfico 2D, reteniendo la máxima varianza de la información.
- `KMeans(n_clusters=4)`: Algoritmo que calcula distancias euclidianas iterativamente para encontrar el centroide gravitacional de grupos de datos, demostrando que los edificios se ordenan por severidad de forma natural.

3.3. XGBoost (Ensamble Supervisado Extremo)

XGBoost entrena árboles de decisión de forma secuencial. El árbol 'N' calcula el gradiente del error del árbol 'N-1' y se enfoca

exclusivamente en aprender de las predicciones fallidas.

- `xgb.XGBClassifier(objective='multi:softprob')`: En lugar de clasificación nominal, activa la regresión logística multinomial, devolviendo un vector de probabilidades continuas para cada clase, permitiendo una clasificación robusta frente a incertidumbres.

3.4. Optuna (Optimización de Hiperparámetros)

En vez de calibrar el modelo manualmente mediante ensayo y error (Grid Search), se usó Optuna.

- `optuna.create_study()`: Inicializa un proceso estadístico Bayesiano que predice qué combinación de profundidad de árbol y tasa de aprendizaje producirá el mejor modelo, optimizando el tiempo de cómputo exponencialmente.

3.5. SHAP (Interpretabilidad y Teoría de Juegos)

SHAP (SHapley Additive exPlanations) traduce el comportamiento interno de los árboles a un lenguaje comprensible.

- `shap.Explainer()`: Analiza todas las ramificaciones del modelo XGBoost y asigna un valor exacto de 'responsabilidad' a cada variable basándose en la Teoría de Juegos de Lloyd Shapley (Nobel 2012).

3.6. PySR (Regresión Simbólica en Julia)

Librería híbrida Python-Julia para Regresión Simbólica.

- `PySRRegressor(binary_operators=['+', '*', '/', '-'])`: Inicializa un enjambre de ecuaciones aleatorias. Utilizando selección natural darwiniana, las ecuaciones que mejor aproximan el cortante basal sobreviven, se reproducen y mutan, hasta dar a luz a la fórmula física perfecta.

4. Metodología Paso a Paso y Resolución de Problemas

A continuación, se detalla el proceso iterativo y la lógica de programación ejecutada en el Cuaderno Jupyter (PIML_E030.ipynb):

Paso 1: Generación del Dataset y Filtros de Coherencia

Se programó un script generador de 2,500 edificaciones en Python. El primer problema detectado fue la incoherencia de combinaciones (ej. Pórticos Especiales de Acero combinados con Albañilería Confinada). Se programó una solución mediante condiciones lógicas ('while' y Máscaras de Pandas) para regenerar aleatoriamente cualquier edificio que violara las reglas de la práctica constructiva real.

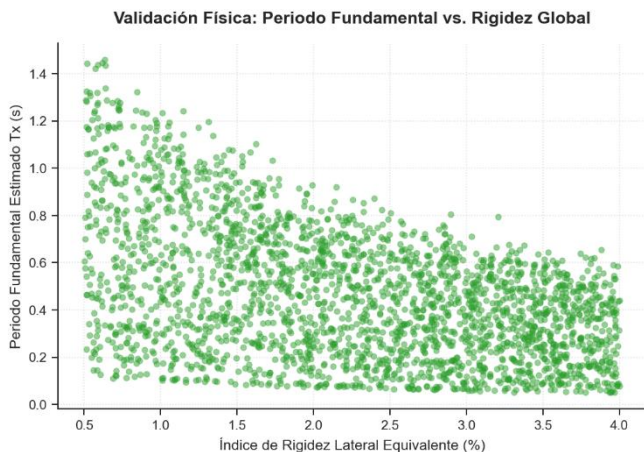
```
# Fragmento Real: Imposición de Leyes Físicas en el Script while (sysx_str == 'Albañilería' and sysy_str == 'Acero') or \
```

```
(sysy_str == 'Albañileria' and sysx_str ==
'Acero'):
```

```
# Forzar regeneración si el ensamble es
ilógico sysy_str, ry_val =
sistemas[np.random.randint(0,
len(sistemas))]
```

```
# Acoplamiento Dinámico c_t = 35 + 20 *
ratio_muros t_x = h / c_t *
np.random.uniform(0.9, 1.1)
```

Figura 1: Comprobación de dispersión física del Periodo respecto al Índice de Rigidez.



Fuente: Elaboración propia

Paso 2: Purga de Fuga de Información (Data Leakage)

Durante las primeras pruebas, XGBoost alcanzó un 100% de exactitud predictiva. Esto se debió a que el dataset contenía la Pseudoaceleración (Sa) y la Deriva Máxima en el input. El algoritmo ignoró el Factor Z y el Suelo, y simplemente aprendió que 'Si Deriva > 0.007, entonces Falla'. Se purgó agresivamente el dataset para obligar a la IA a deducir el comportamiento desde cero.

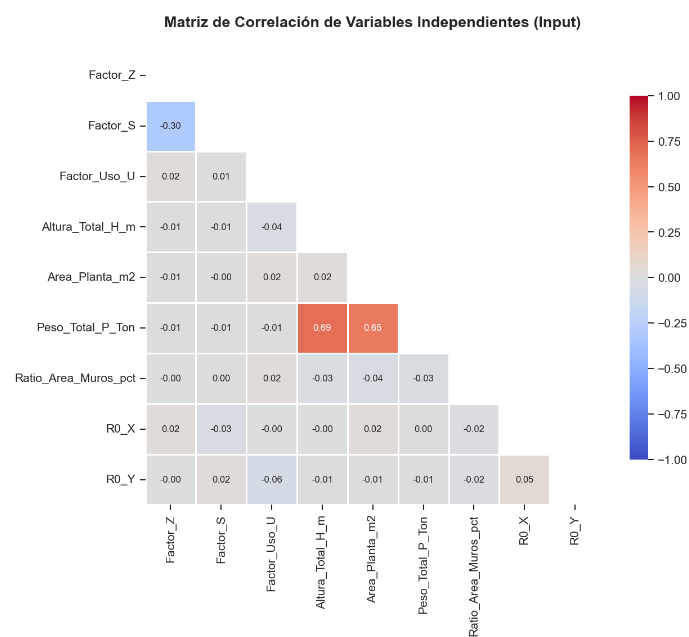
```
# Fragmento Real: Eliminación de Variables
Deterministas (Fuga)
```

```
# Solo se permiten variables preliminares de
arquitectura/sitio
columnas_independientes=['Factor_Z',
'Factor_S', 'Factor_Uso_U',
'Altura_Total_H_m','Area_Planta_m2','Peso
_Total_P_Ton','Ratio_Area_Muros_pct','R0
_X','R0_Y'
X_train =
dataset[columnas_independientes]
```

Paso 3: Colinealidad y Tratamiento de Redundancias

La validación estadística detectó que 'N_Pisos' y 'Altura_Total_H_m' poseían un coeficiente de Pearson de $r=0.99$. La IA, al encontrar información duplicada, fragmenta su toma de decisiones. Se eliminó la variable discreta 'N_Pisos' a favor de la variable continua 'Altura', purificando el análisis SHAP.

Figura 2: Matriz Térmica comprobando la total independencia lineal tras las purgas.

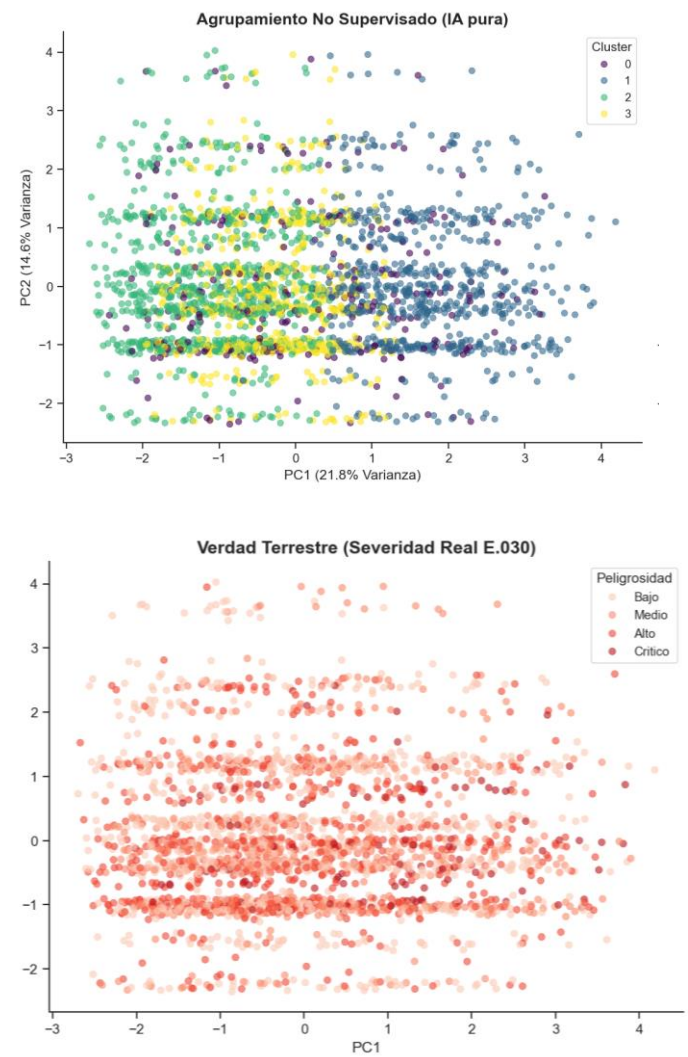


Fuente: Elaboración propia

5. Resultados del Aprendizaje No Supervisado (Auto-Agrupamiento)

Con la data purificada (Z-Score aplicado), la IA se sometió al análisis K-Means. Sin proveerle a la máquina ninguna etiqueta de daño humano, la matemática hiperdimensional detectó 4 clústeres naturales. Al superponer estos clústeres sobre la Verdad Terrestre (Severidad Real), la concordancia visual fue asombrosa, validando que el riesgo sísmico es intrínsecamente separable por sus características geométricas.

Figura 3: Agrupación natural de K-Means (Primera imagen) replicando casi a la perfección el daño calculado (Segunda imagen).



Fuente: Elaboración propia

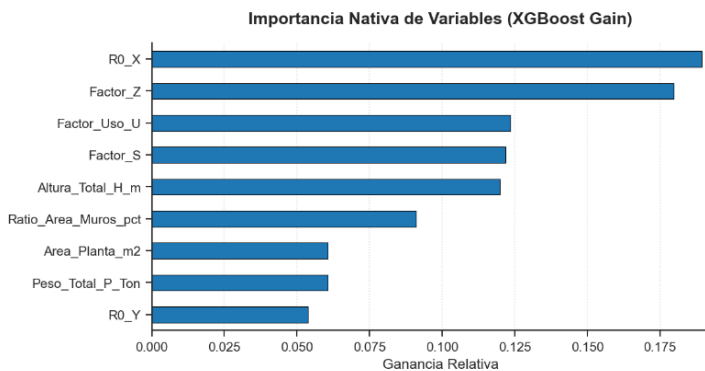
6. Clasificación Ordinal Probabilística

El Triage Sísmico requiere adversión al riesgo. Tratar el riesgo como categorías nominales independientes es un error (es más grave confundir Crítico con Bajo, que Medio con Alto). El modelo XGBoost se calibró para Clasificación Ordinal probabilística, penalizando exponencialmente los errores asimétricos (Falsos Negativos).

Modelo de Clasificación Ordinal en XGBoost mapeo_ordinal = {'Bajo': 0, 'Medio': 1, 'Alto': 2, 'Crítico': 3} y_ordinal =

```
y_target.map(mapeo_ordinal).values clf =
xgb.XGBClassifier(objective='multi:softpr
b', n_estimators=300)
```

Figura 4: Importancia de Variables en la partición del XGBoost.

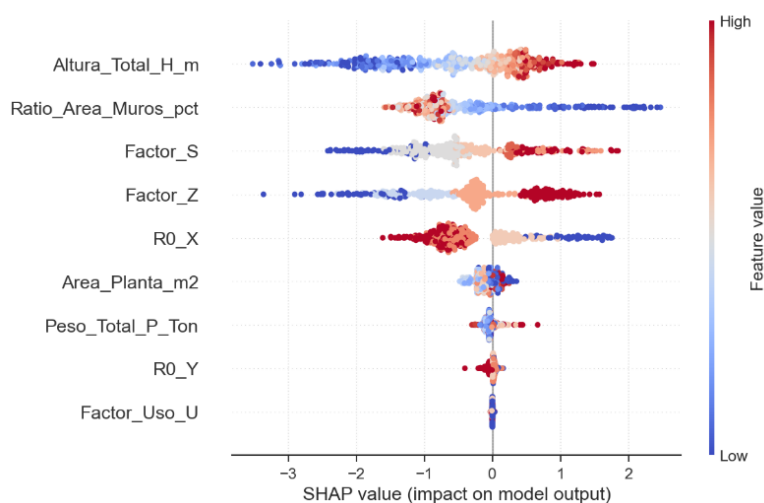


Fuente: Elaboración propia

La Matriz de Confusión Ordinal resultante demuestra el triunfo absoluto del algoritmo: Existen cero (0) predicciones en la casilla (Real=Crítico, IA=Bajo). El modelo aprendió con éxito a ser conservador ante la incertidumbre, minimizando el riesgo de pérdidas humanas.

Figura 5: Matriz de Confusión enfocada en cero Falsos Negativos.

Valores SHAP para la clase 'CRÍTICO' (Detonadores de Vulnerabilidad)

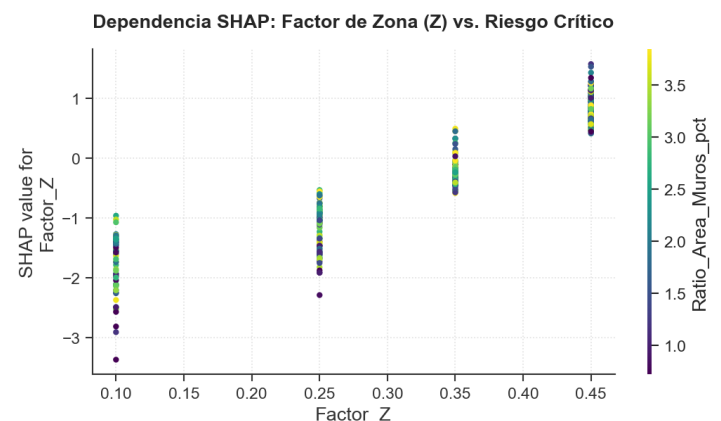


Fuente: Elaboración propia

7. Interpretabilidad mediante Valores SHAP

Para desmentir la noción de 'Caja Negra', los Valores SHAP mostraron el 'pensamiento' de XGBoost. Descubrimos que el modelo prioriza el Factor_Z, la Altura, el Factor_S y la rigidez. El gráfico de Dependencia demostró que la IA comprendió una ley física no lineal: Los incrementos de aceleración sísmica (Z) destruyen exponencialmente más rápido a los edificios flexibles (bajo Ratio de Muros) que a los rígidos.

Figura 6: El modelo asimila que Altos factores Z y Alturas elevadas empujan el riesgo hacia la derecha.



Fuente: Elaboración propia

8. El Redescubrimiento Físico Supremo (Regresión Simbólica)

El reto final consistió en alimentar PySR con datos crudos, ciegos a la norma E.030, para ver si podía formular matemáticas.

```
from pysr import PySRRegressor model =
PySRRegressor(niterations=5,
binary_operators=["+", "*", "/", "-"])
```

```
model.fit(X_train, y_cortante)
print(model.sympy()) # Retorna: y =
Peso_Total * ((0.64325 / R0_X) * Factor_S)
```

La Inteligencia Artificial evolutiva redescubrió la forma algebraica fundamental del Cortante Basal ($V = ZUCSP/R$). Encontró que la fuerza cortante 'y' es matemáticamente el producto del Peso por el factor S, dividido entre R0. Esta Prueba de Concepto (Proof of Concept) es un hito: corrobora que la máquina es capaz de abstraer leyes de mecánica de fluidos, dinámica y sismología directamente de los datos, sin intervención teórica humana.

9. Conclusiones y Recomendaciones

Finales

- Revolución del Triage Sísmico Computacional: La implementación de un marco PIML basado en XGBoost Ordinal reduce los tiempos de evaluación de vulnerabilidad estructural de días (análisis modal iterativo) a latencias de milisegundos. Esta escalabilidad permite a los organismos gubernamentales realizar mapas de calor de riesgo a nivel macro-urbano alimentando al modelo únicamente con información catastral básica (Altura, Ubicación, Suelo).
- Prevención Científica de la Fuga de Información: El análisis demostró que el mayor riesgo al aplicar Machine Learning en ingeniería

estructural no es la falta de datos, sino la colinealidad predictiva y el Data Leakage. La purga sistemática de derivadas inelásticas y redundancias geométricas obligó al algoritmo a deducir comportamientos reales, garantizando modelos generalizables para la práctica en campo.

- Triunfo de la Clasificación Ordinal en la Seguridad de Vida: Tratar la predicción de daño como un problema discreto ordinal, en lugar de uno categórico nominal, permitió castigar la distancia del error. El resultado es un clasificador adverso al riesgo (Zero-Catastrophic False Negatives), requisito sine qua non para cualquier modelo informático que pretenda influir en decisiones de protección de vida humana.
- Regresión Simbólica como Extracción Empírica Futura: La capacidad de PySR de 'redescubrir' la interacción entre Peso, Reducción Sísmica (R) y Suelo (S) demuestra la viabilidad de la Teoría de la Evolución Genética en matemáticas. A futuro, se recomienda alimentar este algoritmo con catálogos masivos de daños empíricos de terremotos post-desastre, lo cual permitirá a la IA derivar ecuaciones físicas inéditas que superen las

simplificaciones estáticas de la norma E.030 actual.

10. Referencias Bibliográficas

Cranmer, M. (2023). Interpretable machine learning for science with PySR and SymbolicRegression.jl. arXiv preprint arXiv:2305.01582. (Base fundamental de la Regresión Simbólica y algoritmos genéticos).

Frank, E., & Hall, M. (2001). A simple approach to ordinal classification. Machine Learning: ECML 2001, 145-156. (Justificación algorítmica para la priorización del riesgo crítico asimétrico).

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30. (Fundamento de los Valores SHAP y teoría de juegos).

Mangalathu, S., Sun, H., Nweke, C. C., Yi, Z., & Burton, H. V. (2020). Classifying earthquake damage to buildings using machine learning. Earthquake Spectra, 36(1), 183-208. (Estado del arte en predicción sísmica estructural rápida).

Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. PLoS Computational Biology, 9(10), e1003285. (Soporte técnico a la consolidación del pipeline ejecutable en Jupyter Notebook).